CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- Capstone Demo Day
  - Weds April 22, 1pm-5pm in the Alumni House
  - See what your classmates have accomplished!
- 1.5 weeks of classes left after today
- You have assignments due April 10, 17
- Questions / discussion about final project?

Today:

Software Architecture

Software Architecture:

The idea that for any given piece of software, the parts fit together according to some well-organized high-level structure

Example:

You're trying to understand a piece of software and you see that it's a collection of composable processing elements that data flows through

Last week we talked about how modularity is the key to building large software

Architecture is just a shorthand for describing different styles of modularity

Why is software architecture important?

1. Super useful shorthand for humans and LLMs. If I tell you “Qt is event-driven” then you already know a lot about how it works!

Why is software architecture important?

2. It tells us at a glance if the software is likely to have certain properties.

E.g. a client-server system is very likely to tolerate client failures well

3. A good choice of architecture makes it easy to make the kinds of changes that we want to support making.

The architecture can remain stable over a long period of time, and many versions

4. A good choice of software architecture helps the humans and LLMs working on the software coordinate with each other

5. The architecture can solidify trust, security, and coordination boundaries, allowing us to scrutinize risky parts of the software.

It can create boundaries where we can enforce policies.

Can LLMs work within existing software architectures?

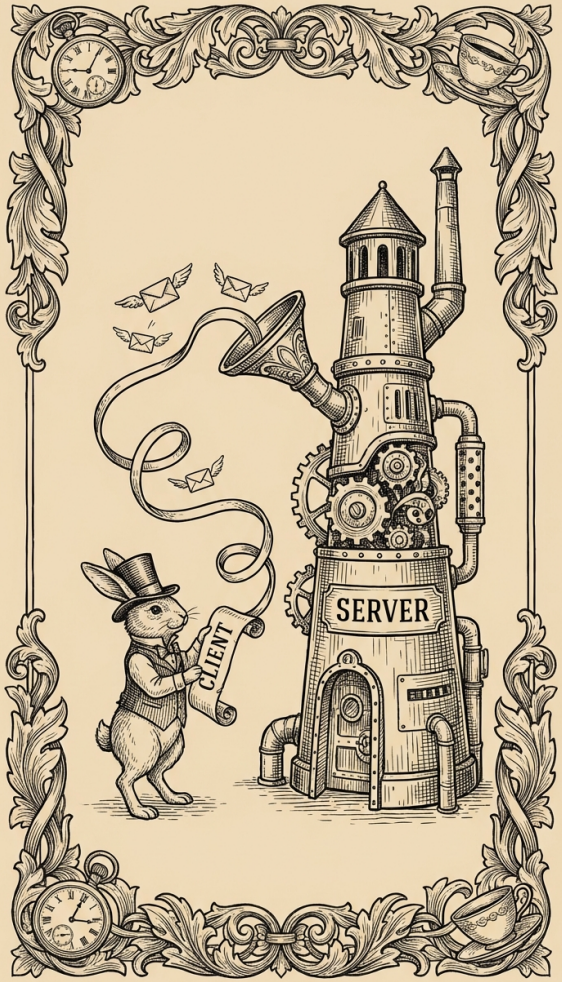
Can LLMs create well-architected software from scratch?

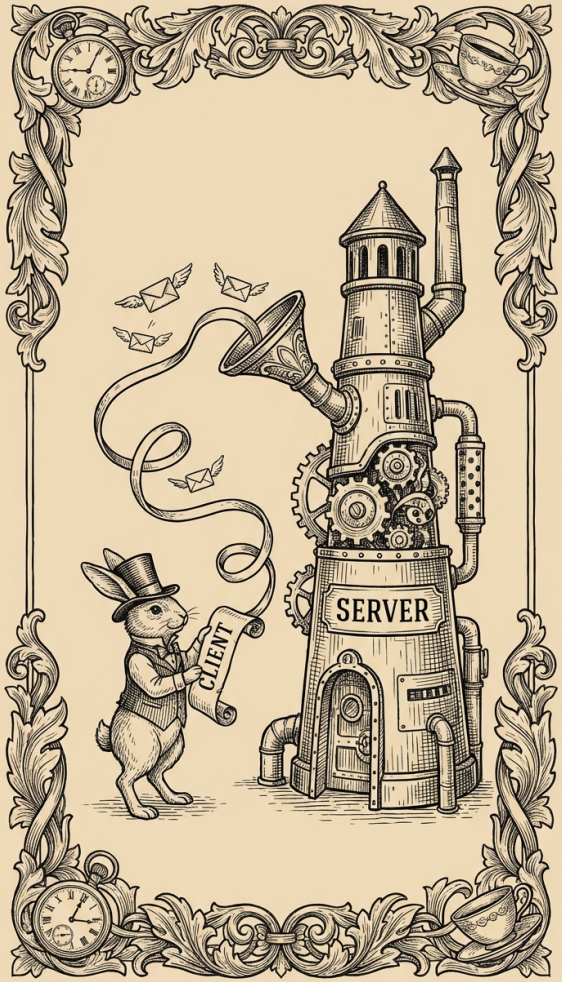
So… what are some examples of software architectures?

Client-Server

There is a persistent, stateful server that runs somewhere, probably in the cloud

It might need a lot of storage, it might need to be fast. Typically this is where the complexity goes





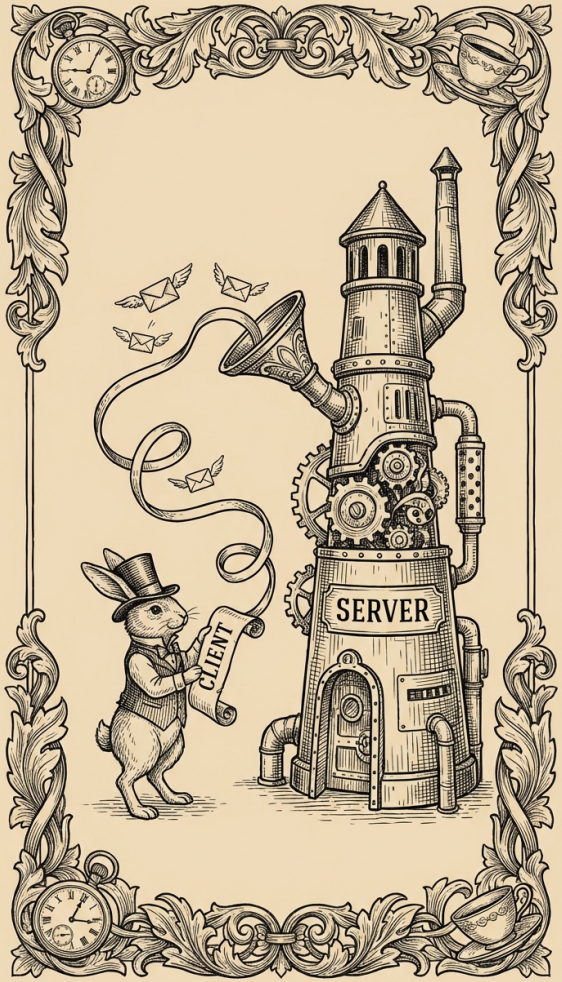
Client-Server

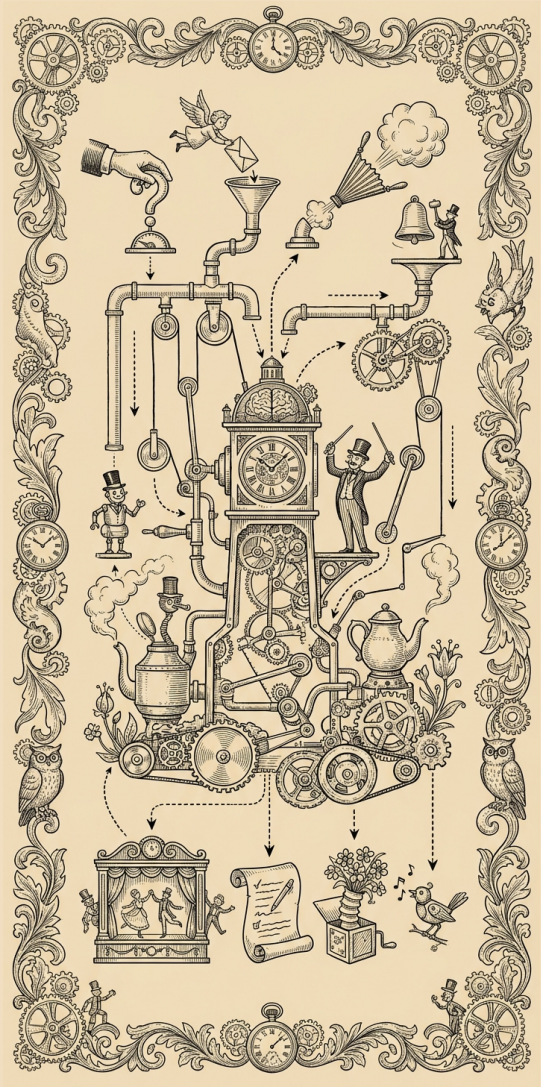
Ephemeral clients connect to the server.

They are simple, untrusted, and usually as close to stateless as possible.

Client-Server examples:

- Databases
- Google Earth
- DHCP
- Like, A LOT of stuff out there is client-server



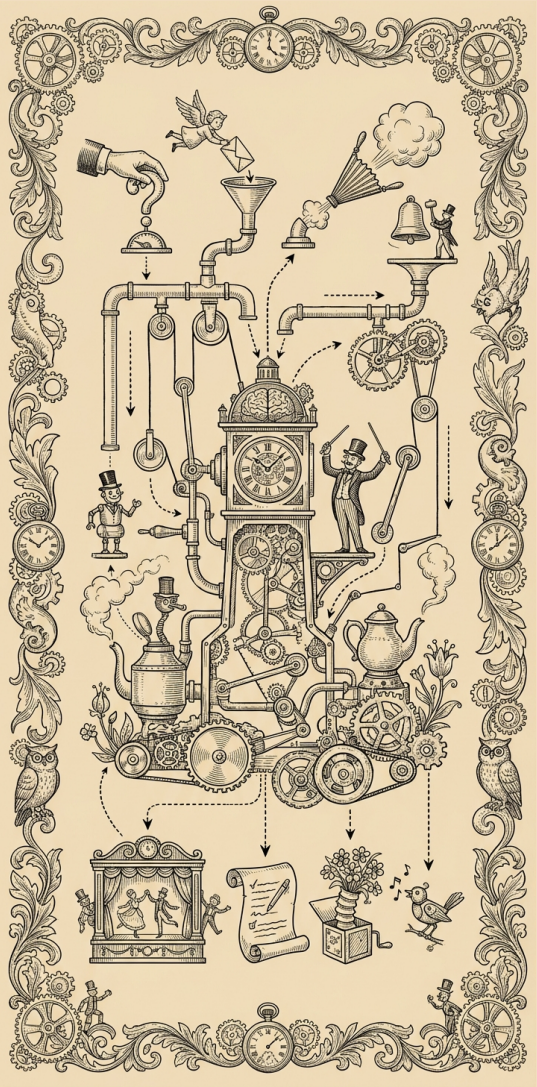


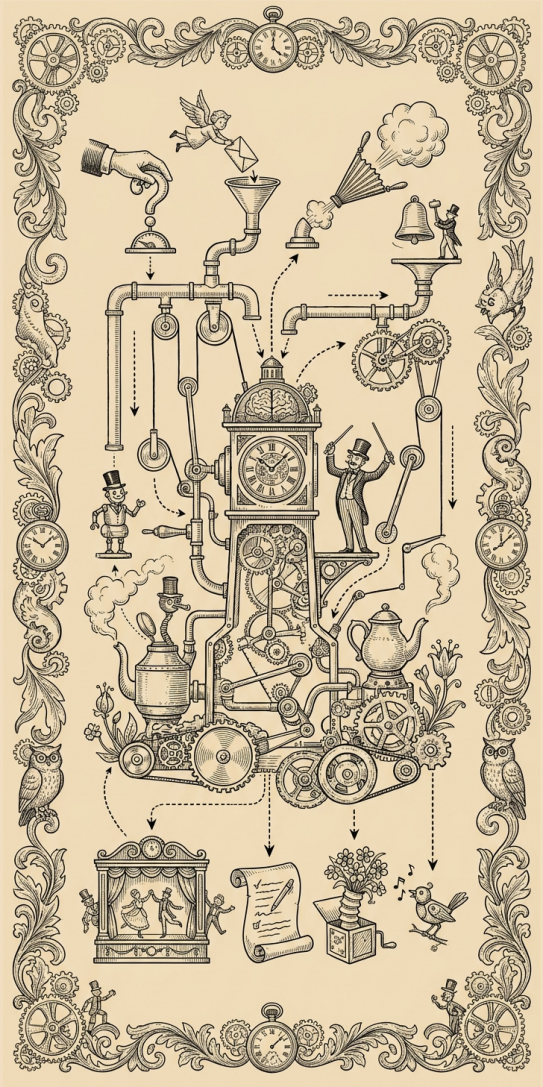
Event-driven software is passive, it does not execute until something in the environment triggers an event, for example:

- User presses a key
- Customer arrives in the store
- Price of soybeans goes below $1

An event handler:

1. Updates program state
2. Optionally takes externally-visible actions
3. Optionally triggers additional events
4. Terminates



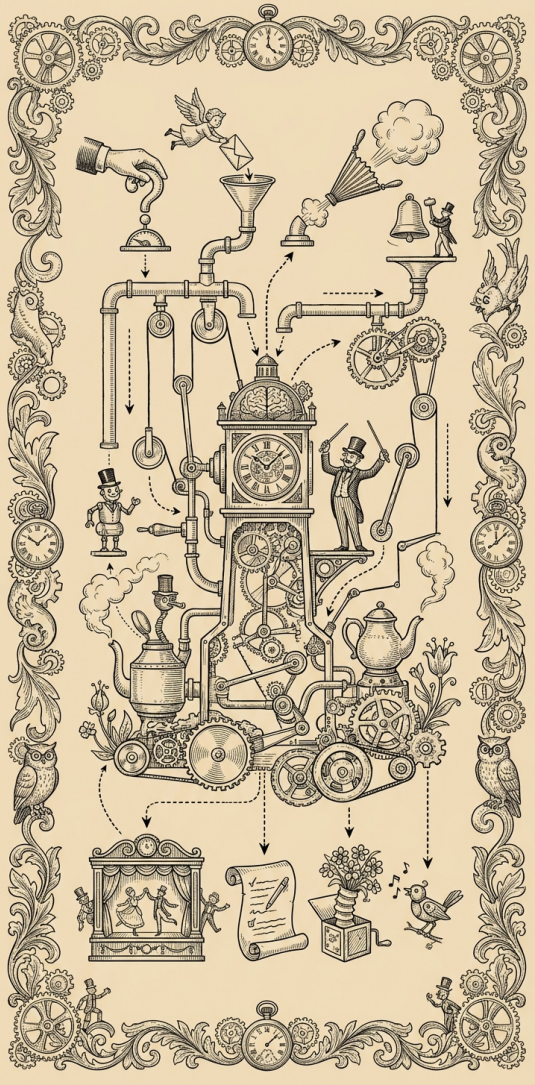


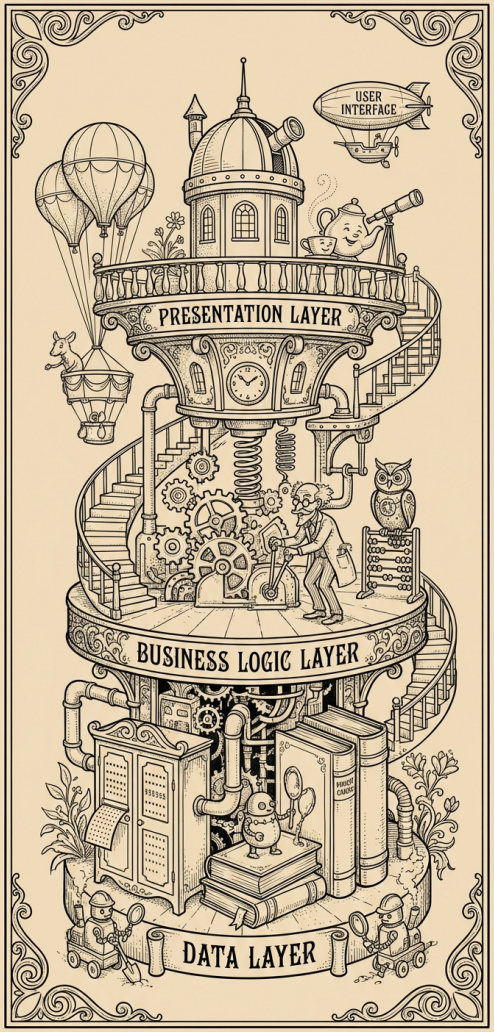
Event handlers might or might not execute concurrently

Event handlers might fire in FIFO order, or priority order, or some other order

Event-driven examples:

- Many GUI applications
  - Including your PyQt-based text editor
- Many embedded systems
  - Raw CPUs are event driven by default



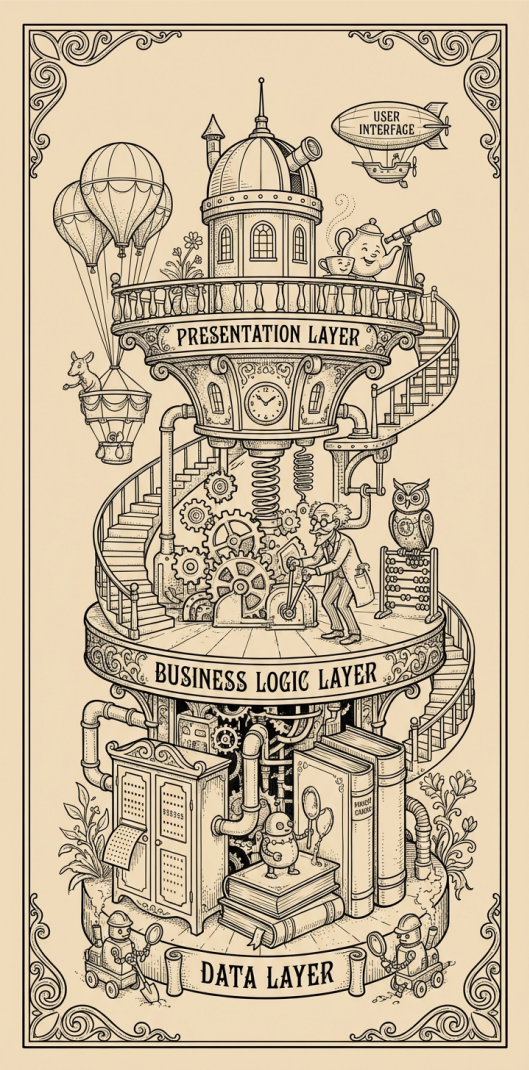


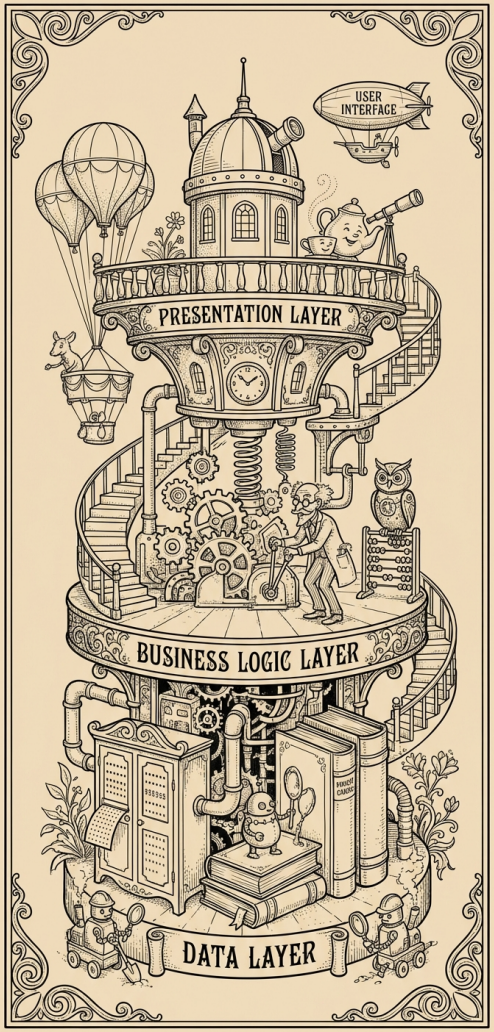
The **layered software architecture** builds a stack of abstractions.

Every layer builds on the one below, and is built upon by the one above.

Layering violations are frowned upon, but sometimes necessary.

They tend to make maintenance and debugging more difficult.





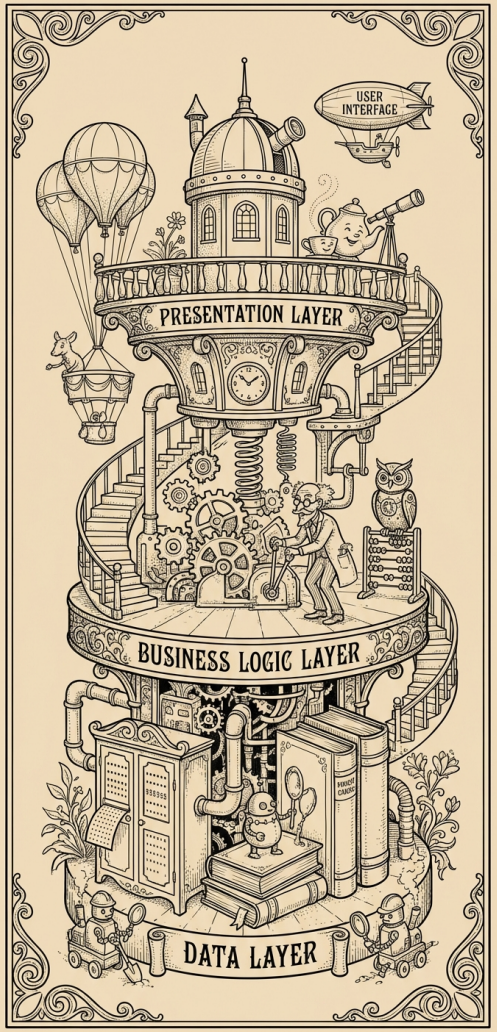
Layered software is super common.

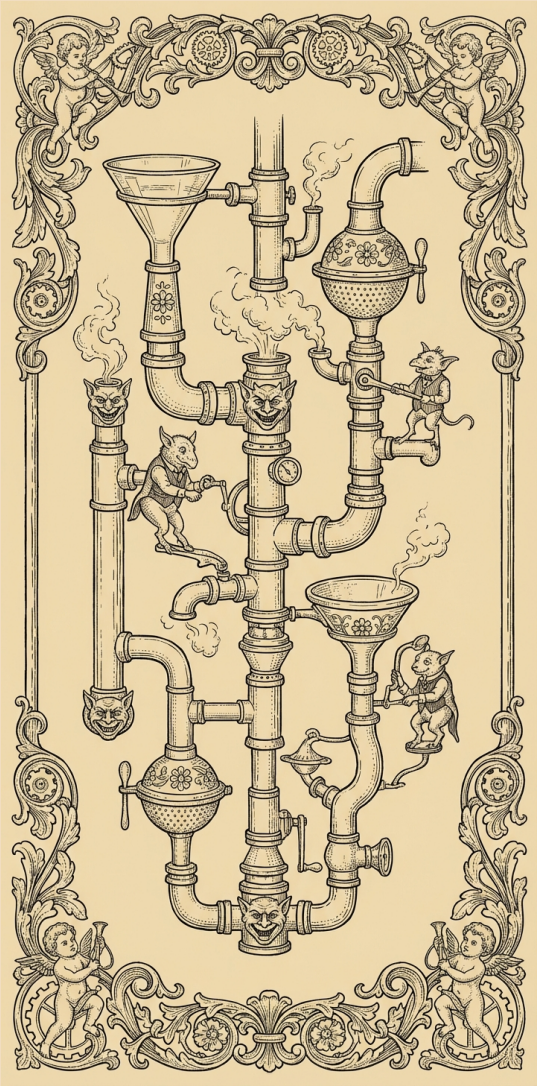
It happens somewhat naturally when we reuse libraries, libraries reuse others, etc.

But we also build layered systems explicitly

Examples of layered software:

- Device abstractions in operating systems
- Text editors
- Web browsers



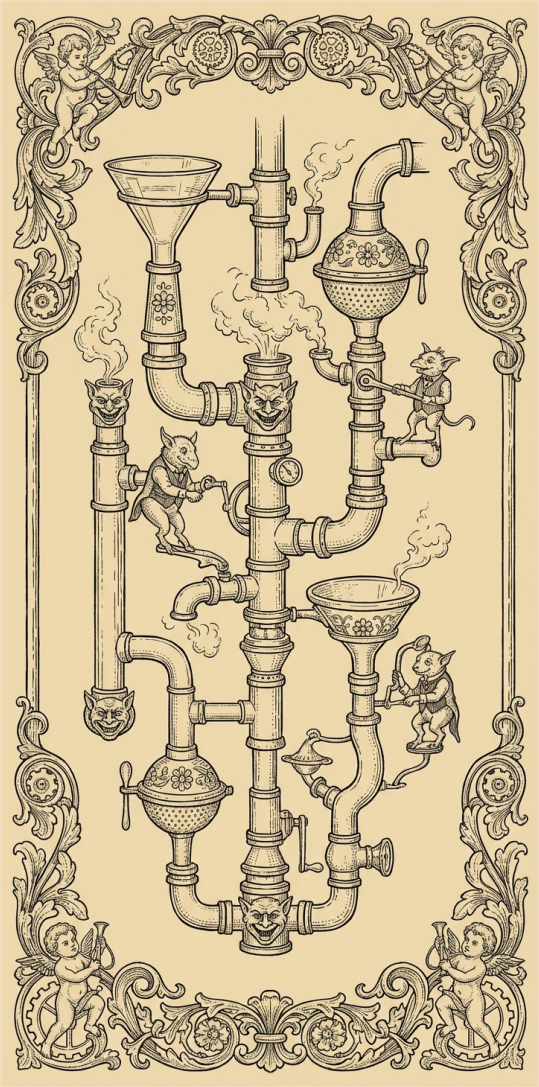


In the **pipe-and-filter** architecture, data flows through a series of processing steps

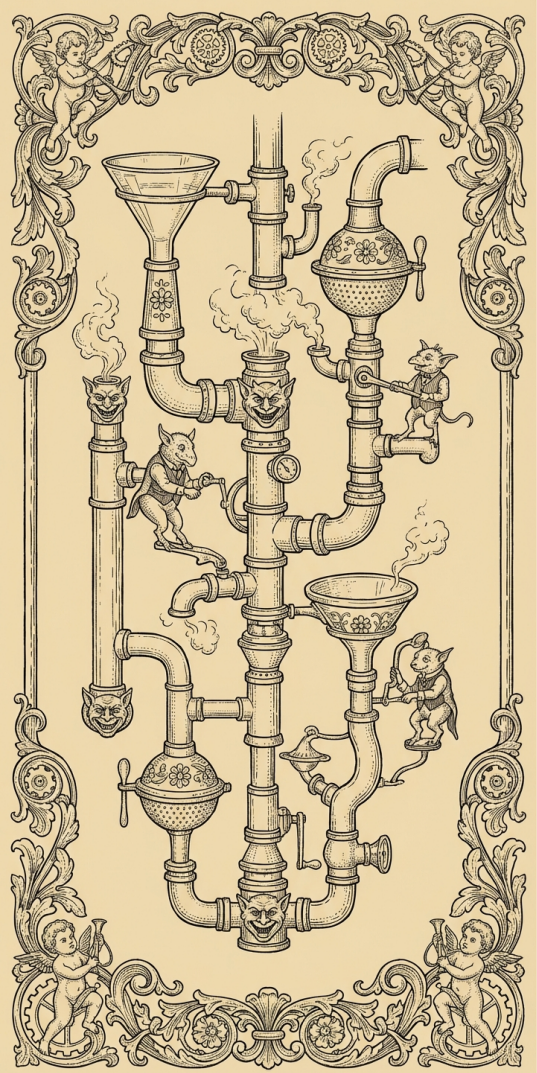
The steps themselves are often stateless

Examples of pipe-and-filter:

- Image processing
- Data analysis
- Machine learning



A key advantage of the pipe-and-filter architecture is that you can easily reorder, swap out, modify filtering elements without affecting how other parts of the pipeline work



This was just a few examples

There are a lot more architectures out there!

When we say that a piece of software has some particular architecture, this is almost never 100% accurate

So:

“ImageMagick is a pipe-and-filter program”

isn’t a lie; it’s just not the complete picture

What happens when we don’t impose an architecture on a piece of software?

The results are often a mess!

This is fine for small software, not OK as the system grows

(GCC example)

You can retrofit software architecture via refactoring

This is a lot of work

But coding agents are likely to change the economics of this

Our recommendation is to start thinking explicitly about software architecture

What examples do you see in software you use?

Can you learn anything that you can apply to your own work?

Also, it's worth trying to get LLMs to work within your project's architecture

Sometimes they recognize the architecture and do this on their own

Sometimes you have to keep telling them